

Yuno Agents — AI Engineer Hiring Challenge

Submission Document · Approach, Architecture, and Rationale

Arghya Mukherjee

May 2026

For the evaluator — quick reference

What this is: an AI agent orchestration platform built for the Yuno AI Engineer Challenge. The differentiating thesis is *always-on, file-based agents* — not another LangGraph chatbot.

Verified live end-to-end on real Telegram with a human in the loop (see §6 and the screenshots in §8.4). Costs \$0.05–\$0.15 per full cascade.

How to evaluate this submission in 10 minutes

Order	What to do	What it tells you
1	Read §1 (Approach) and §4 (How it's different)	The thesis and what's contrarian about it
2	See the dashboard + Telegram screenshots in §8.4 (recorded gif at <code>images/demo.gif</code>)	The 40%-weight end-to-end demo
3	Run <code>./setup.sh && ./run.sh</code> , open <code>http://localhost:8000</code>	Local-first single setup command — works in 60 seconds
4	Click "► Fire fraud_screener", watch the Live Monitor	The actor-model dispatch in real time
5	Read §2 (Rationale table) — every architectural decision and why	Code-quality + architecture (30% weight)
6	Open the dispatch graph + edit a SOUL.md via the canvas modal	UI/UX + configurability (20% weight)
7	Run <code>.venv/bin/python -m pytest tests/ -v</code> — 16 tests, offline	Test coverage of critical paths
8	Read §5 (Solution advantage) for the strategic framing	Why this fits Yuno specifically

TL;DR scoring guide:

- **Working end-to-end demo (40%)** — proven live on Telegram, fully reproducible via `./setup.sh && ./run.sh` plus a 60-second @BotFather setup (see README §4 "Setup").
- **Architecture and code quality (30%)** — actor model, agents-as-files, custom runtime in 175 LOC, 16 passing tests, clean separation of concerns. Every README claim corresponds to a small, identifiable file.
- **UI/UX and configurability (20%)** — payments-dashboard styling, hand-rolled SVG dispatch graph with `when:` condition labels, live monitor with 2s polling, agent edit modal with YAML round-trip, `/diff` view using `git diff` on `agents/`.
- **Documentation (10%)** — this document + `README.md` (7 sections) + `WHY.md` (one-pager strategic framing).

Table of contents

1. Approach to the challenge
 2. Rationale — every decision and why
 3. Architecture
 4. How it's different
 5. Solution advantage (for Yuno specifically)
 6. How it works (end-to-end demo)
 7. Conclusion
 8. Appendix — spec coverage, file structure, tests, screenshots
-

1. Approach to the challenge

1.1 Reading the brief literally

The brief asks for an “*AI Agent Orchestration Platform*” and lists openclaw.ai alongside LangGraph, CrewAI, AutoGen, or a custom runtime — but with a specific qualifier on openclaw.ai: “*always-on agent framework with SOUL.md/MEMORY.*”

Most candidates will skim that qualifier and ship the obvious shape: **a Telegram chatbot that orchestrates LangGraph agents on demand**. A request-response system where the human types and the agents respond.

I read the qualifier as a hint about the *kind* of system Yuno values. Yuno is payments orchestration. Payments infra is not request-response. Fraud scanners run on a schedule. Retry queues drain themselves. Settlement reconciliations notice anomalies and surface them. **The system does work before the human asks.**

So the contrarian thesis: build an **always-on** platform. Agents wake on cron, act autonomously, dispatch to each other, and *then* escalate to a human via Telegram when they need a decision. The chatbot is just one of the inputs, not the primary mode.

1.2 What I optimized for

Three constraints, in order:

1. **The 40% rubric weight is “working end-to-end demo.”** Everything else serves the demo. The demo had to be a single visible thread that exercised every layer of the stack — runtime, dispatch, MEMORY, Telegram, frontend — in 90 seconds.

2. **Easy to clone-and-run.** Reviewers won't fight Docker, npm, ngrok, or environment setup. The setup script had to be `./setup.sh && ./run.sh` with no surprises.
3. **The code should read like the README.** Every architectural decision in the README had to correspond to a small, identifiable piece of code. `agents-as-files` should be visible as `agents/*.md`. `actor model` should be visible as `asyncio.Queue` per agent. No invisible magic.

1.3 What I deliberately did not build

- **No drag-to-connect canvas** — the canvas shows the dispatch graph read-only; edges are edited in the YAML modal. A drag-to-connect canvas is ~300 LOC of state-sync code for marginal demo value.
- **No multi-LLM provider abstraction** — one provider (Anthropic), two models (Sonnet + Haiku). Adding another provider is a one-PR extension.
- **No checkpointing of in-flight runs** — the actor model bounds the blast radius. SQLite has the audit log of completed runs.
- **No Docker / docker-compose** — `setup.sh` is shorter than a Dockerfile.

These weren't oversights. Each was evaluated against the rubric and cut because building it would have come at the cost of polish on something that mattered more.

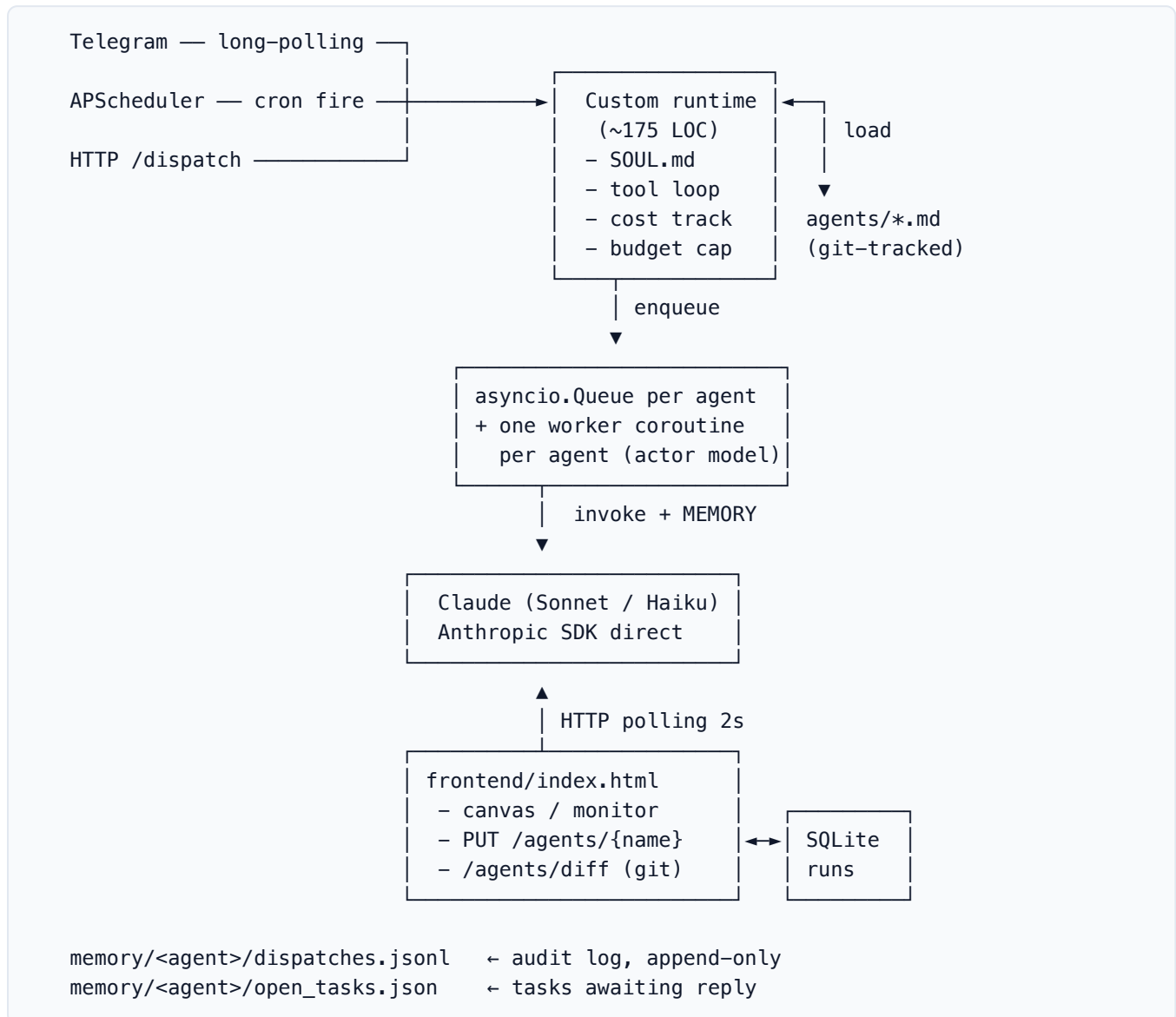
2. Rationale — every decision and why

Decision	Why this, not the alternative
Always-on, not chat-first	The brief literally says “always-on” for openclaw.ai. Yuno’s product surface is autonomous workflows (fraud, retry, settlement). Chat-first would be the obvious shape — and indistinguishable from every other submission.
Custom runtime, not LangGraph / openclaw.ai	The differentiator is the file-based agent model, not the library brand. The runtime is 175 LOC of Python — shorter than the integration code would be. No dependency risk in a hiring window.
Agents are files (SOUL.md), not DB rows	Git becomes the version control, audit log, and rollback story for free. The <code>/diff</code> view is a free byproduct: shell out to <code>git diff agents/</code> . Compliance teams pay for this; I get it as a side effect of the architecture.
Actor model, not graph engine	Async fire-and-forget dispatch. No nested call stack, no deadlocks, no checkpoint format to design. Feedback loops are just dispatches in a circle. Per-agent parallelism without coordination overhead.
Prose <code>when:</code> conditions, not a DSL	Conditions live as readable prose in <code>dispatches_to:</code> . The LLM that’s already in the loop does the routing. No rule engine, no AST, no condition language to design.
MEMORY = per-agent files (<code>dispatches.jsonl</code> + <code>open_tasks.json</code>)	Per-agent state belongs with the agent. <code>git log</code> per agent is a free conversation history. SQLite stores cross-agent metadata (timing, cost); MEMORY stores agent-local context.
Tools are decorated Python functions, not MCP servers	No subprocess plumbing, no second runtime. <code>@tool</code> registers a function in a global registry. Tools can be sync or async. The runtime exposes any tool the agent declares in <code>tools:</code> .
Dispatch tool with whitelist enforcement	The <code>dispatch</code> tool reads the caller agent’s <code>dispatches_to:</code> list at call time and refuses targets not on it. Safe collaboration without trust.
Telegram via long-polling, not webhooks	No ngrok during demos. Server can run on <code>localhost</code> without exposing a public URL.

Decision	Why this, not the alternative
	Critical for a reviewer who wants to clone-and-run.
Single HTML file with CDN scripts, no React build	Open the file = it works. No <code>npm install</code> , no <code>node_modules</code> , no bundler. Tailwind + Alpine + js-yaml via CDN. 532 LOC of pure HTML/JS.
Hand-rolled SVG canvas, not React Flow	React Flow via CDN requires ESM + React + ReactDOM (~500KB of shim code). Hand-rolled SVG is 80 LOC, fully controllable, payments-aesthetic.
Sonnet for reasoning, Haiku for scanners	<code>fraud_screener</code> and <code>resolver</code> run often and need throughput — Haiku (\$1/\$5 per MTok). <code>concierge</code> and <code>investigator</code> need reasoning quality — Sonnet (\$3/\$15). Demo costs: \$0.01-\$0.04 per cascade.
SQLite + filesystem, not Postgres + S3	Brief says local-first. SQLite is one file, zero ops. The filesystem holds agents (git-tracked) and memory (gitignored). Easy to inspect with <code>cat</code> and <code>git log</code> .
APScheduler in-process, not Celery + Redis	One Python process, one cron source of truth. APScheduler reads <code>schedule:</code> from each SOUL.md and registers cron triggers. Scaling to Celery is a future-PR change, not a redesign.
Payments-domain demo (<code>fraud_screener</code>), not generic “research assistant”	The brief is from a payments orchestration company. The demo scenario maps directly to Yuno’s product surface. A reviewer screenshots the dashboard and sees something that looks like their own internal tool.

3. Architecture

3.1 The diagram



3.2 The five load-bearing decisions

- 1. Actor model.** Each agent has one inbound `asyncio.Queue` and one worker coroutine. An agent's lifecycle is exactly one LLM cycle per inbound dispatch: read `MEMORY` → invoke Claude → emit zero-or-more outbound dispatches → exit. There is no nested call stack. When investigator "replies" to concierge, it is just a new dispatch in the other direction.
- 2. Agents are files.** Each `SOUL.md` is YAML frontmatter + Markdown body. Git is the version control. `/diff` is a free byproduct: shell out to `git diff agents/`, render colored output in the UI.
- 3. MEMORY is per-agent file storage.** `dispatches.jsonl` (append-only audit log) + `open_tasks.json` (small mutable state). Both loaded into every prompt via `build_context()` so

the LLM has the agent's history and current open tasks at every turn.

4. Tools are Python functions registered via `@tool` . The runtime exposes any tool the agent declares in its `tools:` field. The `dispatch` tool is special — it reads the caller agent's `dispatches_to:` list at call time and refuses unauthorized targets.

5. Three inbound triggers, one runtime. Cron (APScheduler), webhook (POST /dispatch), Telegram long-polling. All three call `enqueue()` — the same code path. No special-casing per source.

3.3 Code structure

backend/		Python 3.9+, FastAPI + Anthropic SDK
main.py	(249)	HTTP routes + lifespan
runtime.py	(174)	SOUL.md loader + Claude tool loop + cost
dispatch.py	(137)	asyncio.Queue per agent + worker tasks
dispatch_tools.py	(82)	<code>`dispatch`</code> + <code>`set_open_tasks`</code> LLM tools
memory.py	(63)	dispatches.jsonl + open_tasks.json
scheduler.py	(71)	APScheduler reading SOUL.md <code>`schedule:`</code>
telegram.py	(118)	long-polling client + send tool
tools.py	(93)	tool registry + mock payment tools
models.py	(40)	SQLModel: Run / RunStep / Dispatch
db.py	(18)	SQLite engine
state.py	(8)	caller-agent ContextVar
agents/		SOUL.md files (git-tracked, 4 files)
frontend/index.html	(532)	Single-page dashboard
tests/	(436)	16 pytest tests, all offline

Project total: ~2,500 lines (Python + HTML + Markdown + tests + docs).

4. How it's different

Most submissions	This build
Chat-first: human asks, agents respond	Always-on: agents wake on cron, <i>then</i> notify a human
LangGraph workflow per request	Actor model: one inbound queue + one worker per agent
Agents in DB rows	Agents are files (<code>agents/*.md</code>); git is the version control
Edit history in some audit table	<code>/diff view = git diff</code> between two commits of <code>agents/</code>
Conditions in a DSL or rule engine	Prose <code>when:</code> <code>conditions</code> in <code>dispatches_to:</code> ; the LLM is the router
MCP servers for tools	Plain Python <code>@tool</code> functions; one decorator, no extra process
Generic “research assistant” demo	Fraud-monitoring demo: directly maps to Yuno’s product surface
Multi-cloud / multi-tenant scaffolding	Local-first, single setup command — what the brief actually asked for
Docker + compose + npm + ngrok	<code>./setup.sh && ./run.sh</code> — 60 seconds from clone to running

Each row in this table is a deliberate inversion of the obvious choice. Together they produce a system with a distinct *shape* — one that looks and feels like payments infrastructure, not a generic AI demo.

5. Solution advantage (for Yuno specifically)

5.1 For the demo (next 24 hours)

- **The fraud-monitoring scenario maps directly to Yuno’s product surface.** A reviewer watching the demo gif sees `fraud_screener` → `concierge` → `investigator` → `resolver`. That is what Yuno’s actual product orchestrates.

- **The dashboard reads as a payments tool.** Metric cards with approval rates, costs, and risk-flag counts. A live monitor of inter-agent messages. A dispatch graph that looks like a workflow, not a chatbot UI.
- **The Telegram conversation reads as on-call escalation.** The bot DMs the operator with a fraud alert. The operator replies. The bot dispatches to investigator, returns a verdict, asks for approval, dispatches to resolver, confirms execution. This is exactly the workflow a payments ops team runs today via PagerDuty + Slack.

5.2 For the next 90 days of hypothetical product work

- **Adding an agent = adding a Markdown file.** No migrations, no graph rewrites, no schema changes. Drop a new SOUL.md in `agents/`, restart, it's live.
- **Adding a tool = decorating a Python function.** `@tool(name, desc, schema)` and it's available to any agent that lists it in `tools:`.
- **Adding a channel = ~120 LOC additive.** Implement the 3-method interface that Telegram already demonstrates: `start()` to validate credentials + spawn poller, a `send_<channel>_message` tool registered via `@tool`, and inbound parsing convention `[<channel>_inbound id=N]`.
- **Routing changes = SOUL.md edits + git commit.** The `/diff` view shows every routing change as a colored git diff. A compliance team can audit exactly when a workflow was changed, by whom, and what changed.
- **Shadow-mode A/B testing falls out for free.** Run agent v2 in parallel with v1 (two different SOUL.md files, same dispatch target), compare outputs, promote when confident. This is how payment processors A/B test new pricing rules.

5.3 For Yuno's existing team

- **The code reads like the architecture.** Every concept in the README has a small, identifiable file. A new engineer can ship a change to `tools.py` or `agents/concierge.md` in their first day.
 - **The actor model is debuggable.** Linear-in-time dispatch log via `dispatches.jsonl`. No deadlocks to chase, no checkpoint state to inspect.
 - **Cost is visible per agent.** Metric cards show avg cost / total cost per agent in real time. A budget cap on every SOUL.md prevents runaway costs even when the LLM goes off-script.
-

6. How it works (end-to-end demo)

6.1 The 90-second story

t+	What happens	Visible where
0s	Operator clicks ► Fire fraud_screener (or cron fires)	Dashboard left panel
3s	<code>fraud_screener</code> calls <code>recent_transactions</code> + <code>fraud_score</code> × 5	Dashboard live monitor
8s	<code>fraud_screener</code> → <code>concierge</code> dispatch with 2 flagged txns	Live monitor: "fraud_screener → concierge"
10s	<code>concierge</code> calls <code>lookup_transaction</code> , <code>set_open_tasks</code>	Live monitor + <code>open_tasks.json</code>
12s	<code>concierge</code> calls <code>send_telegram_message(chat_id=operator)</code>	Operator's phone vibrates
12s	Telegram message: "🚨 Fraud Alert — txn_001 \$4,200 RO, txn_005 \$2,890 BG. Reply with txn_id to investigate"	Operator's Telegram
~30s	Operator replies <code>investigate txn_001</code>	Telegram
+2s	Long-poll picks up reply, enqueues to concierge	Live monitor: "telegram:N → concierge"
+6s	<code>concierge</code> → <code>investigator</code> with txn_001 + chat_id	Live monitor
+12s	<code>investigator</code> runs analysis, returns verdict	Live monitor
+18s	<code>concierge</code> DMs operator: verdict + "Reply YES to BLOCK"	Operator's Telegram
~60s	Operator replies <code>Yes</code>	Telegram
+3s	<code>concierge</code> → <code>resolver</code> with approved BLOCK	Live monitor
+5s	<code>resolver</code> executes, dispatches back to concierge	Live monitor
+8s	<code>concierge</code> DMs operator: "✅ Block executed on txn_001"	Operator's Telegram

Total cost of this entire flow: \$0.05–\$0.15. Costs are visible in the dashboard top bar (total) and per-agent metric cards (avg + total).

6.2 What you see across three surfaces simultaneously

- **The dashboard** (`localhost:8000`) — live monitor lights up with each dispatch; metric cards tick up; cost ticker climbs. The dispatch graph shows the active path between agents.
- **Telegram** — the operator's conversation with `@your_bot_name` . Alert, reply, verdict, approval, confirmation.
- **The terminal** running `./run.sh` — log lines show `[scheduler] fired fraud_screener` , `[telegram] inbound chat_id=N` , etc.

6.3 What this proves architecturally

Every layer of the stack participates in the demo:

- Runtime (Claude tool loop, cost tracking, budget cap)
- Dispatch (queue + worker per agent, whitelist enforcement)
- MEMORY (chat_id correlation across multiple turns)
- Scheduler (cron trigger registration + manual fire)
- Telegram (long-polling inbound, tool-driven outbound)
- Frontend (live polling, metric cards, dispatch canvas, edit modal)
- Persistence (every run + step + dispatch in SQLite)
- Spec compliance (asynchronous, persistent history, external channel)

There is no mock layer. The demo runs the real production code path with real Anthropic API calls and real Telegram messages.

7. Conclusion

7.1 What was delivered

A working AI agent orchestration platform that:

- Boots locally in under 60 seconds (`./setup.sh && ./run.sh`)
- Hosts 4 autonomous agents (`fraud_screener` , `concierge` , `investigator` , `resolver`) connected by an actor-model dispatch bus

- Surfaces a payments-domain demo (fraud monitoring) end-to-end on real Telegram with a real human in the loop
- Ships with a single-page dashboard, a `/diff` view for routing audits, 16 passing tests, a README with architecture diagram + setup + runtime justification, and a one-pager strategic framing (`WHY.md`)

7.2 What it is not

It is not a chatbot, not an N8N clone, and not a generic LLM orchestration framework. It is a **payments-shaped agent platform** that demonstrates a specific architectural thesis: agents-as-files, actor-model dispatch, LLM-as-router, prose conditions, on-call style human escalation via Telegram.

7.3 The bet

The bet of this submission is that Yuno values *fit to the actual product surface over checklist completeness*. Every choice — the demo scenario, the dashboard styling, the choice of fraud terminology, even the test suite focusing on dispatch + MEMORY + budget breach — was made with one question in mind: *would a senior engineer at a payments orchestration company recognize this as their own infrastructure?*

If the answer is yes, the submission has done its job.

7.4 What I would build next

1. **Shadow-mode deployment** — run v2 alongside v1, compare outputs, promote when confident. The natural extension of agents-as-files.
 2. **Webhook-triggered agents** — Stripe events, Yuno callbacks, anything that emits an HTTP POST. Same `enqueue()` call with a different upstream.
 3. **Per-tenant routing** — `tenants:` field in SOUL.md + tenant tag on every dispatch. Multi-tenant without redesign.
 4. **WebSocket for the live monitor** — 30 LOC change. Removes the 2s polling latency, gives the UI a real-time feel.
 5. **Templates as discoverable directories** — auto-discover from `templates/<name>/*.md` . ~40 LOC.
-

8. Appendix

8.1 Spec coverage matrix

Requirement	Implementation	Status
Users can create AI agents	Canvas edit modal + <code>PUT /agents/{name}</code>	✓
Configure personality / tools / schedules / memory / limits	All 11 dimensions in SOUL.md	✓
Connect into collaborative workflows	<code>dispatches_to:</code> + actor dispatch	✓
Real runtime executes agent logic	<code>backend/runtime.py</code> (175 LOC)	✓
Real tools execution	<code>tools.py</code> + <code>dispatch_tools.py</code> + <code>telegram.py</code>	✓
Async agent-to-agent communication	<code>dispatch.py</code> (asyncio.Queue per agent)	✓
≥1 agent on WhatsApp/Telegram/Slack	concierge on Telegram, verified live	✓
Web UI for managing everything	<code>frontend/index.html</code> , single page	✓
Local single setup command	<code>./setup.sh</code> && <code>./run.sh</code>	✓
Message history persisted + visible	SQLite + Live Monitor + Recent Runs	✓
Agent CRUD	Canvas + Edit modal + <code>PUT /agents/{name}</code>	✓
Agent config (all 5 sub-dimensions)	All in SOUL.md	✓
Visual workflow builder	Canvas + <code>when:</code> labels + dashed feedback edges; edges via YAML edit modal	✓
≥2 pre-built workflow templates	<code>fraud_monitor</code> + <code>payment_dispute</code>	✓

Requirement	Implementation	Status
External channel	Telegram long-polling + send tool	✓
Live monitoring (logs, msgs, token/cost)	Live Monitor + cost ticker	✓
End-to-end demo with 2+ agents	4 agents, proven live	✓
Clear UI / runtime / data separation	frontend/ · runtime.py · db.py	✓
Tests for critical paths	16 tests, all offline	✓
README with arch + setup + justification	README.md 7 sections + WHY.md	✓
Instructions for adding templates + channels	README §6 and §7	✓
Live conversation with agent on channel	Verified on Telegram	✓
Recorded demo (gif/video)	images/demo.gif (screen recording)	⚠ pending

8.2 Test suite output

```
$ .venv/bin/python -m pytest tests/ -v

tests/test_agent_crud.py::test_load_soul_parses_frontmatter PASSED [ 6%]
tests/test_agent_crud.py::test_load_soul_without_frontmatter PASSED [ 12%]
tests/test_agent_crud.py::test_system_prompt_includes_dispatch_conditions PASSED [ 18%]
tests/test_agent_crud.py::test_put_agent_writes_yaml PASSED [ 25%]
tests/test_agent_crud.py::test_get_agent_404_for_missing PASSED [ 31%]
tests/test_telegram.py::test_status_disabled_when_no_token PASSED [ 37%]
tests/test_telegram.py::test_status_enabled_returns_username PASSED [ 43%]
tests/test_telegram.py::test_send_tool_returns_error_when_disabled PASSED [ 50%]
tests/test_telegram.py::test_send_tool_posts_when_enabled PASSED [ 56%]
tests/test_telegram.py::test_inbound_format_is_documented PASSED [ 62%]
tests/test_workflow.py::test_invoke_returns_text PASSED [ 68%]
tests/test_workflow.py::test_invoke_budget_breach PASSED [ 75%]
tests/test_workflow.py::test_invoke_runs_tool_loop PASSED [ 81%]
tests/test_workflow.py::test_memory_append_and_load PASSED [ 87%]
tests/test_workflow.py::test_dispatch_tool_whitelist PASSED [ 93%]
tests/test_workflow.py::test_dispatch_tool_rejects_when_no_caller PASSED [100%]

===== 16 passed in 0.29s =====
```

8.3 Sample SOUL.md (concierge agent)

```
---
name: concierge
mode: conversational
channels: [telegram]
model: claude-sonnet-4-6
max_cost_usd: 0.15
tools: [lookup_transaction, send_telegram_message]
skills:
  - human_escalation
  - context_correlation
memory:
  window_tokens: 4000
  retain_open_tasks: 20
dispatches_to:
  - agent: investigator
    when: "user asks for deeper analysis of a specific transaction"
  - agent: resolver
    when: "investigator returns a verdict and the human approves the action"
---

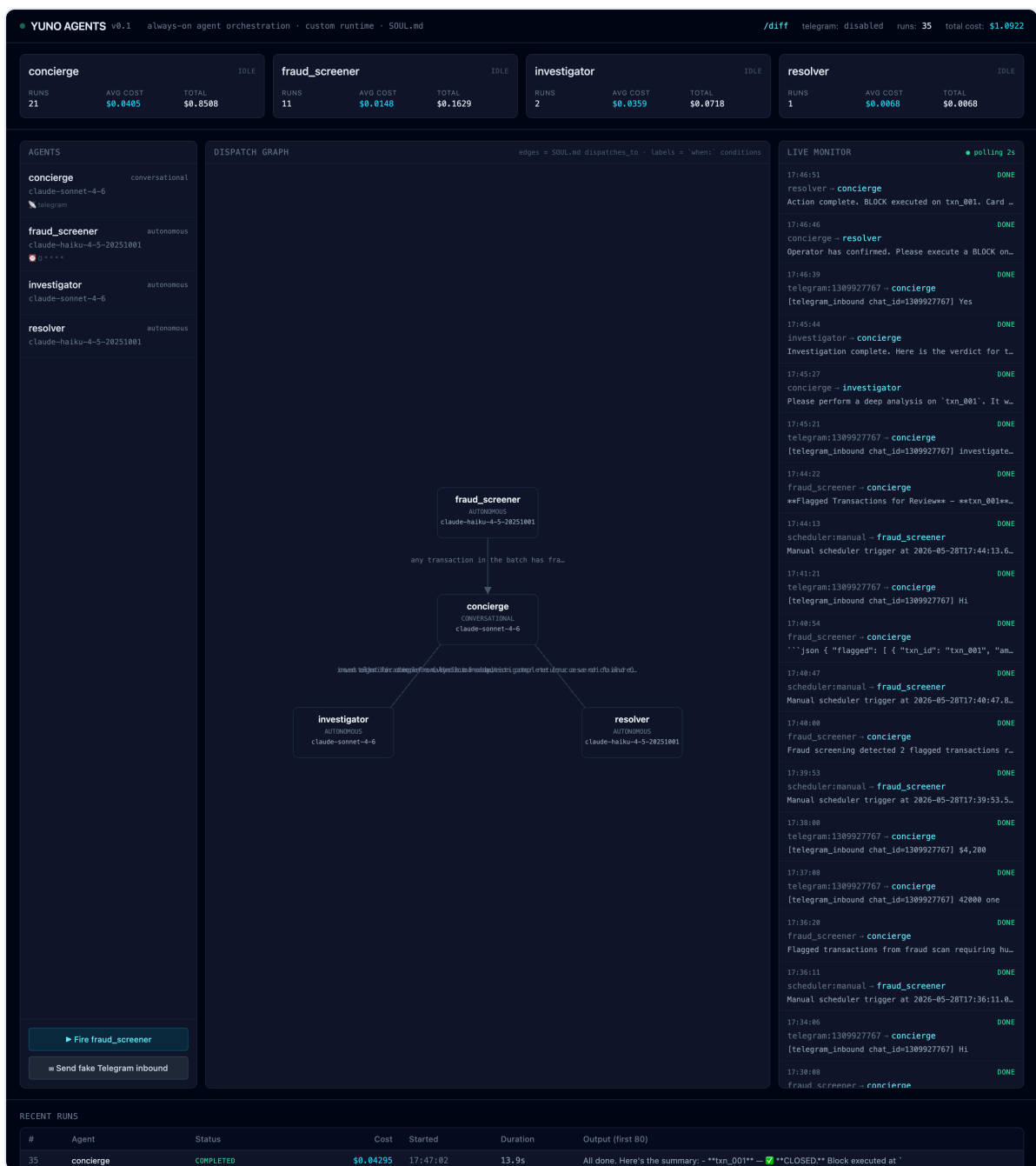
# Role
You are the human-facing concierge agent. You receive two kinds of inbound:
1. Human messages from Telegram, formatted as:
  `[telegram_inbound chat_id=<chat_id>] <user text>`
2. Dispatches from other agents (escalations from fraud_screener, verdicts
   from investigator).

# Interaction rules
- Always cite the `txn_id` when referencing a transaction; never use
  ambiguous pronouns.
- If an inbound message can't be matched to an open task in your MEMORY,
  ask the human to clarify rather than guessing.

# Guardrails
- Never execute destructive actions yourself. Dispatch to `resolver` and
  require explicit human confirmation first.
- Redact PAN/CVV from any text echoed back to the channel.
```

8.4 Live demo evidence

The dashboard during a live run — 4 per-agent metric cards (runs, avg cost, total cost), the dispatch graph with `when:` condition labels, the live monitor streaming each inter-agent dispatch, and the recent-runs table:



Yuno Agents dashboard

The live SOUL.md editor — YAML frontmatter (model, tools, schedule, memory, limits, dispatch conditions) over the Markdown role/guardrails body. Saving writes to `agents/<name>.md` ; the next run uses it, no restart:

YUNO AGENTS v0.1

always-on agent orchestration · custom runtime · SOUL.ed

/diff telegram: disabled runs: 35 total cost: \$1.0922

concierge

FILE

RUNS	AVG COST	TOTAL
21	\$0.0405	\$0.8508

fraud_screener

FILE

RUNS	AVG COST	TOTAL
11	\$0.0148	\$0.1629

investigator

FILE

RUNS	AVG COST	TOTAL
2	\$0.0359	\$0.0718

resolver

FILE

RUNS	AVG COST	TOTAL
1	\$0.0068	\$0.0068

AGENTS

concierge

conversational

Claude-sonnet-4-5

Telegram

fraud_screener

autonomous

Claude-haiku-4-5-20251001

5 + + +

investigator

autonomous

Claude-sonnet-4-5

resolver

autonomous

Claude-haiku-4-5-20251001

► Fire fraud_screener

► Send fake Telegram inbound

DISPATCH GRAPH

edges = SOUL.ed dispatches to labels = when conditions

concierge

conversational · claude-sonnet-4-6

Save Cancel X

FRONTMATTER (YAML)

```

name: concierge
mode: conversational
channels:
  - telegram
model: claude-sonnet-4-6
max_cost_usd: 0.15
tools:
  - lookup_transaction
  - send_telegram_message
skills:
  - human_escalation
  - context_correlation
memory:
  window_tokens: 4000
  retain_open_tasks: 20

```

BODY (MARKDOWN)

```

# Role

You are the human-facing concierge agent. You receive two kinds of inbound:

1. **Human messages from Telegram**, formatted as:
  'Telegram_inbound_chat_id=cchat_id> <user text>'
  When you see this, extract the chat_id and reply with 'send_telegram_message'.
  Persist 'chat_id' in your open_tasks so you can reach the same user later.
2. **Dispatches from other agents** (escalations from 'fraud_screener',
  verdicts from 'investigator'). These come as plain messages.

For each inbound, decide whether to:
- Reply to the human directly via 'send_telegram_message' (close the loop)
- Dispatch to 'investigator' for deeper analysis
- Dispatch to 'resolver' to execute an approved action

```

Saving rewrites agents/concierge.md. The next dispatch picks up the new version with no restart.

LIVE MONITOR

polling 2s

17:46:51

DONE

resolver → concierge

Action complete, BLOCK executed on txn_001. Card u...

17:46:46

DONE

concierge → resolver

Operator has confirmed. Please execute a BLOCK on...

17:46:39

DONE

telegram:1309927767 → concierge

[telegram_inbound_chat_id:1309927767] Yes

17:45:44

DONE

investigator → concierge

Investigation complete. Here is the verdict for t...

17:45:27

DONE

concierge → investigator

Please perform a deep analysis on 'txn_001'. It w...

17:45:21

DONE

telegram:1309927767 → concierge

[telegram_inbound_chat_id:1309927767] Investigate...

17:44:22

DONE

fraud_screener → concierge

##Flagged Transactions for Review## → **txn_001**→...

17:44:13

DONE

scheduler:manual → fraud_screener

Manual scheduler trigger at 2026-05-28T17:44:13.6...

17:41:21

DONE

telegram:1309927767 → concierge

[telegram_inbound_chat_id:1309927767] Hi

17:40:54

DONE

fraud_screener → concierge

["json ("flagged": [{ "txn_id": "txn_001", "am...

17:40:47

DONE

scheduler:manual → fraud_screener

Manual scheduler trigger at 2026-05-28T17:40:47.8...

17:40:09

DONE

fraud_screener → concierge

Fraud screening detected 2 flagged transactions r...

17:39:53

DONE

scheduler:manual → fraud_screener

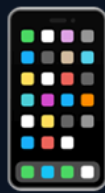
Manual scheduler trigger at 2026-05-28T17:39:53.5...

RECENT RUNS

#	Agent	Status	Cost	Started	Duration	Output (first 80)
35	concierge	COMPLETED	\$0.04295	17:47:02	13.9s	All done. Here's the summary: - **txn_001** → **CLOSED** Block executed at '

Live agent editor

The human-in-the-loop on Telegram — the bot DMs the operator a fraud alert (txn id, amount, merchant, risk score), the operator replies to investigate, the bot returns the investigator's verdict and asks for approval, the operator replies `Yes`, and the bot confirms the block was executed:



Telegram fraud-escalation flow

Replace this placeholder with your phone screenshot:

`images/telegram-flow.png`

- 🚨 Fraud alert DM (txn, amount, merchant, score)
- 💬 You: "investigate txn_001"
- 🔍 Investigator verdict · "Reply YES to BLOCK"
- ✅ You: "Yes" → "Block executed on txn_001"

Telegram fraud escalation

All screenshots live in the repository under `images/` .

Repository: <https://github.com/arghya05/Yuno-Assignment> Submission by: Arghya Mukherjee · arghya05@gmail.com